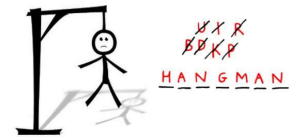




Computational Problem Solving I

CPET-121

Project #4 – HangPerson



Learning Objectives:

The intent of this programming project is to allow students the opportunity to demonstrate their ability to solve problems using C++ programming. This project will focus on file I/O, hierarchical file structure and string manipulation in the implementation of the game Hangman.

Program Description:

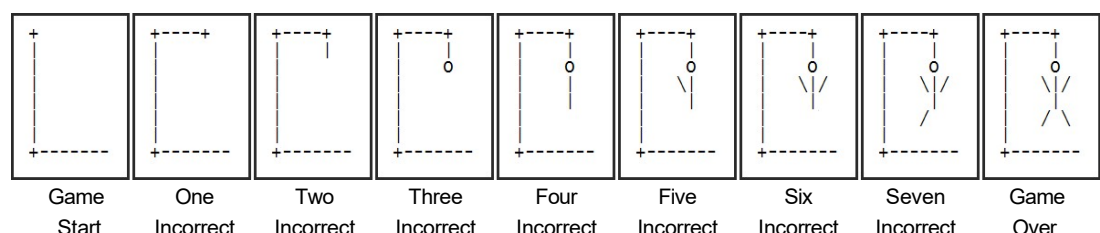
In this project, you will write a C++ program that simulates playing the game **HangPerson**.

Design Specifications:

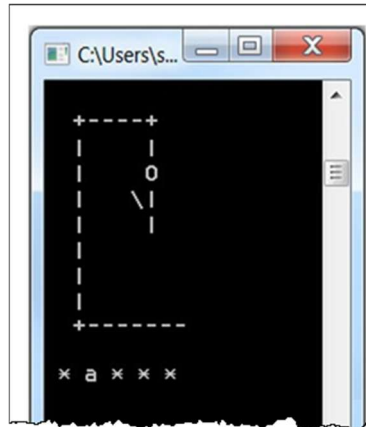
Requirements:

To receive full credit for the implementation, your **HangPerson** game **must** meet the following design specifications:

- This program must contain a **main()** program and four functions: **selectWord()**, **displayGallow()**, **displayWord()**, and **gameInput()**. Each of these must be coded in their own respective CPP source file.
- The function proto-types, and any common global constants, **must** be contained within a header file named **hangPerson.h**.
- Below are the specifications for each of the functions and how they interact with **main()** :
 - Each time the program is run the **selectWord()** function will select a word randomly from a list of words contain within the file **words_1.txt**. This file, which is created by you, consists of a minimum of **thirty** words of **varying lengths**.
 - Note: Your 30 words all need to be 4-8 characters in length
 - No letter may appear more than once in any word,
 - All the letters must be lower case.
 - This function is called by **main()** and returns the selected word to **main()** as a string.
 - The function **displayGallow()** displays the hangman's gallows (see below). This function is called by **main()** and accepts an integer for the total number of incorrect guesses. This number will range from zero (0), representing the start of the game, to eight (8) representing the end of the game. Use the following graphics (or similar) for the game output.



- The function **displayWord()** is called by **main()** and is passed a string that represents the word the user is trying to guess. Initially this string will have an asterisk (*) in place of each of the missing letters. As the letters are correctly guessed, the asterisk will be replaced (in main) with the actual letter. A typical output is shown below, the user is trying to guess a five-letter word, they have had a few incorrect guesses (note the gallows) and has guessed one correct letter ('a') in the second position.



- The function **gameInput()** is called by **main()** and is passed a list (array, vector, or string) of previously guessed letters. This function prompts the user to enter a letter, verify the letter has not already been guessed and return both the guessed letter and the updated list to **main()**. If the user enters a letter that has already been guessed, inform the user and re-prompt for another letter.
 - Note, this re-prompt does **NOT** count as a missed guess.
- The **main()** program controls the flow of the code by looping through each of the above functions until the unknown word is correctly guest or until the gallows it completed (i.e. 8 missed guesses). Along the way, **main()** also needs to keep track of the number of missed guesses, the list of previously guessed characters and the string containing the correct guesses with the asterisks. Additionally, as the game is being played, **main()** shall log the results of each turn into a file named **results_1.txt**. Below are two examples, one for a win and one for a loss.

<pre> The chosen word is uncle. The character i was not found! The error count is 1. The character e was found! The character a was not found! The error count is 2. The character o was not found! The error count is 3. The character u was found! The character k was not found! The error count is 4. The character c was found! The character n was found! The character l was found! The user successfully found the word. </pre>	<pre> The chosen word is final. The character e was not found! The error count is 1. The character i was found! The character o was not found! The error count is 2. The character u was not found! The error count is 3. The character a was found! The character c was not found! The error count is 4. The character p was not found! The error count is 5. The character x was not found! The error count is 6. The character k was not found! The error count is 7. The user failed to find the word. </pre>
---	---

- Below is an example of the game being played and the expected user interactions.
 - Note: The display does not need to look **EXACTLY** like that shown, use your own judgment on what you think looks best.
 - First, an 'a' was guessed (fig #1).



Figure #1

- Since 'a' **was in** the unknown word, the second asterisk was replaced with an 'a' and the hangperson's gallows did not change (fig #2).



Figure #2

- Second, a 'j' was guessed (fig #2).
- Since 'j' **was not** in the unknown word, the crossbeam was added to the hangperson's gallows (fig #3).
- Third, a 'p' was guessed (fig #3).



Figure #3

- Since 'p' **was not** in the unknown word, the rope was added to the hangperson's gallows (fig #4).



Figure #4

- Play continue until either the unknown word is correctly guessed (fig #5) or the number of guesses (8) is exhausted (fig #6)



Figure #5



Figure #6

Grading:

Your grade for this design project is based on the following four elements:

1) Code Execution (40%):

A complete solution that meets all the code design specifications as assessed by the instructor building and running the submitted code.

2) Source Code File (25%):

A copy of all your *.cpp and *.h files for your final design.

- All submissions will be compiled and run on Visual Studio. If you developed your code using another IDE, please indicate so when you submit your source code.
- Please refer to the **C++ Style Guide** for proper formatting, commenting and variable naming.
- **DO NOT SUBMIT the Visual Studio Directory nor the project *.sln file.**

3) Program Operation Documents (25%):

- **A series** of screen shots showing a **win**.
- A print-out of the **results_1.txt** file created by this win.
- **A series** of screen shots showing a **loss**.
- A print-out of the **results_1.txt** file created by this loss.
- A print-out of your text file containing **the 30 words** you created for this project.

4) Code Review Presentation (10%):

A presentation (i.e. PowerPoint, Google Slides, etc.) for your Code Review Presentation.

This presentation shall contain the following components:

- A brief description of your code, main and all functions.
- Sample program outputs. Be sure to demonstrate that the two encoded files you created produce the correct output.
- A review of your testing philosophy.
- Summary of problems encountered & lessons learned while completing this design
- Please see the *Sample Design Presentation* posted to myCourses for more details.

Note: Late projects are penalized 10% per day they are late.